

Detecting Fraud in Online Transactions: A Machine Learning Approach

Student: Nguyen Quoc Trong Nghia (s3343711)
Supervisor: Dr. Nguyen Thien Bao

Agenda

1. Problem & Motivation
2. Research Questions
3. Literature Review & Related Work
4. Dataset
5. Methodology
6. Results & Discussion
7. Limitations & Future Work
8. Conclusion
9. Q&A

Problem & Motivation

- Fraud detection is challenging due to:
 - Severe class imbalance
 - High dimensional data
 - Fraud pattern change over time
 - False Positive (FP) vs False Negative (FN) trade off
 - Multiple evaluation objectives

How can we detect fraud more effectively?

Traditional Approaches & Limitations

- **Traditional approaches:**

- Velocity rules
- Amount thresholds
- IP rules
- Repeated failure rules
- Mismatch rules
- Blocklists

- **Limitations:**

- Depend on predefined conditions
- Difficult to capture complex patterns
- Fixed rules may become outdated

Machine Learning for Fraud Detection

- **Machine learning (ML) approaches**
 - Learn patterns from historical transaction data
 - Capture relationships across multiple features
 - Detect patterns beyond predefined rules
 - Support probabilistic fraud prediction
- **Performance depends on**
 - Feature representation
 - Feature reduction
 - Model selection
 - Class imbalance handling

Research Questions

- **RQ1 - Feature Engineering**
 - How does feature engineering affect fraud detection performance?
- **RQ2 - Feature Reduction**
 - How does feature reduction affect dimensionality and performance?
- **RQ3 - Model comparison**
 - Which ML models perform best across different evaluation metrics and settings?
- **RQ4 - Class Imbalance**
 - How do imbalance handling strategies affect performance and classification errors?

Literature Review & Related Work

Class Imbalance [1]-[3]

- **SMOTE & Random Undersampling**

- ✓ Improve minority class learning
- ✗ Results vary across models, datasets and metrics

Feature Reduction [4]-[7]

- **Feature Selection & Filtering:**

- ✓ Reduce sparse, redundant and weakly informative features
- ✗ Feature engineering and reduction are combined → difficult to distinguish the effect

ML Models [1], [2], [8]-[11]

- ✓ Different model families capture different patterns
- ✗ Performance varies with data representation, class distribution and evaluation settings

Evaluation [2], [12]

- ✓ Temporal evaluation better reflects prediction on later transactions
- ✗ A single metric may provide an incomplete view under class imbalance

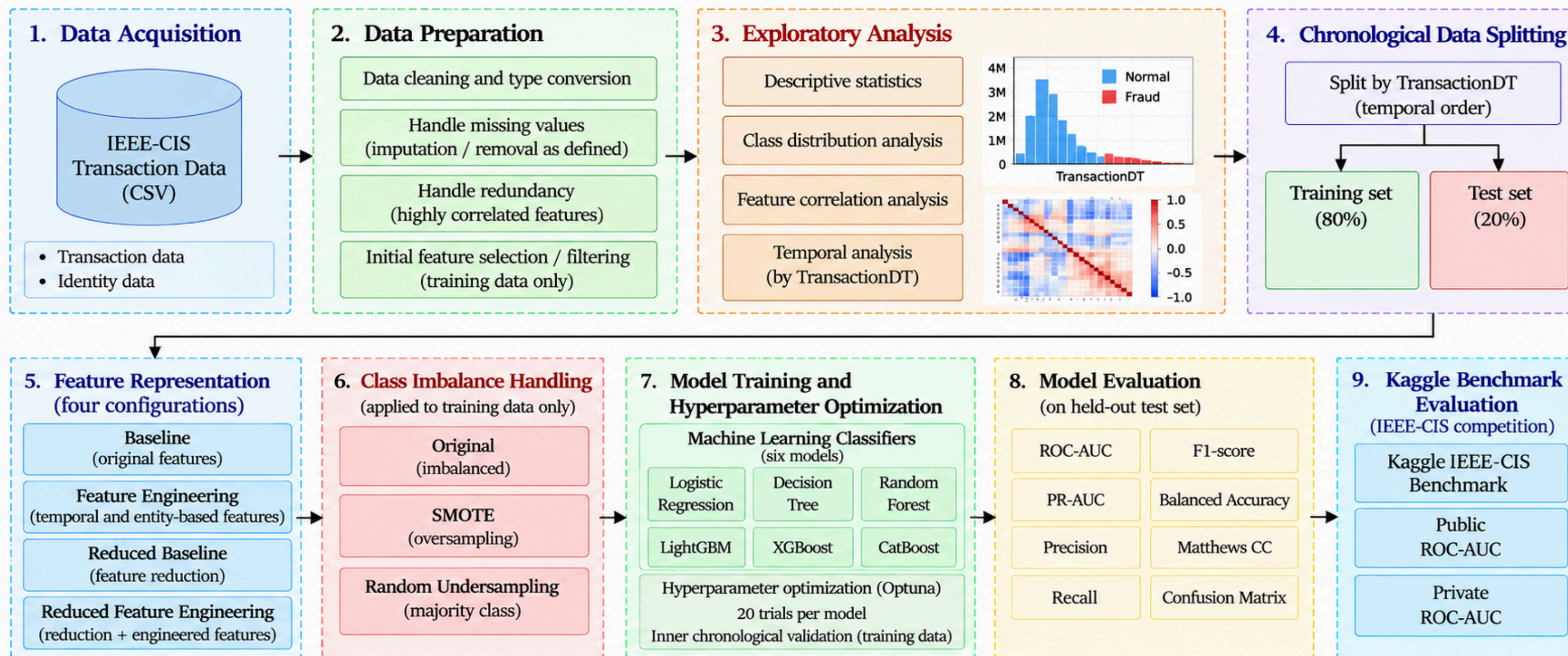
IEEE-CIS Fraud Detection[13]

- Provided by Vesta Corporation
- Real anonymized e-commerce transactions.
- Rich transaction and identity features
- Designed for fraud detection

Dataset - Overview		Feature Group	Columns	Description	Data Type	Example Values	Missing value
		Transaction	TransactionID, TransactionDT, TransactionAmt	Transaction information	Numerical	2987000 86400 68.50	No
Property Value		Product	ProductCD	Product category	Categorical	W,C,R,H,S	No
		Card	card1–card6	Payment card attributes	Card1,2,3,5: Numerical Card4,6: Categorical	150 visa/ mastercard debit/credit	Partial
		Address	addr1, addr2	Anonymized address	Categorical	170.0 87.0	Yes
		Distance	dist1, dist2	Distance and temporal relationship features	Numerical	19.0 30.0	Yes
		Email	P_emaildomain, R_emaildomain	Purchaser & recipient email domains	Categorical	gmail.com hotmail.com	Yes
		C Columns	C1–C14	Counting features	Numerical	1 2 30	No
		D Columns	D1–D15	Time delta features	Numerical	0 1 NaN	Yes
		M Columns	M1–M9	Matching indicators	Categorical	T F NaN	Yes
		V Columns	V1–V339	Vesta engineered numerical features	Numerical	0.0 1.0 NaN	Yes
		Device	DeviceType, DeviceInfo id_30 - id_33	Device information	Categorical	desktop mobile	Yes
		Identity	id_01 - id_38	Identity, authentication features (anonymized)	Mixed	Numeric Categorical	Yes
		Transactions	590,540				
		Identity Records	144,233				
		Target Variable	isFraud (0 = Legitimate, 1 = Fraud)				
		Transaction Features	394				
		Identity Features	41				
		Total Features (after merge)	434+				
		Primary Key	TransactionID				
		Fraud rate	0.03499				
		Identity + Transaction	434 Features (after merge)				

Research Design

Proposed Experimental Framework for Fraud Detection Evaluation



Feature Representation

Baseline: 432 → 342 features

Feature Engineering: 439 → 348 features

Engineered Features:

- uid — composite entity identifier
- uid2 — extended entity identifier
- DT_* — temporal features derived from TransactionDT

	Original Features	Reduced Features
No Feature Engineering	Baseline: Original transaction features	Reduced Baseline: Feature reduction
Feature Engineering	Feature Engineering: Original + temporal and entity features	Reduced Feature Engineering: Engineered + feature reduction

Feature Reduction

- Remove features with $\geq 90\%$ missing values
- Remove highly correlated features ($|r| > 0.98$)
- Remove low information gain features ($IG < 0.001$)

Reduced Feature Sets

- Baseline: 432 $\rightarrow$ 342
- Feature Engineering: 439 $\rightarrow$ 348

Class Imbalance Handling

- **Original**
 - Keep original class distribution
 - No resampling
- **Synthetic Minority Over-sampling Technique (SMOTE) [3]**
 - Oversample minority class
 - Generate synthetic fraud samples
- **Random Undersampling [1],[2]**
 - Reduce majority class
 - Remove legitimate samples

ML Models & Hyperparameter Optimization

Models:

- Logistic Regression (LR)
- Decision Tree (DT)
- Random Forest (RF) [8]
- XGBoost (XGB) [9]
- LightGBM (LGBM) [10]
- CatBoost (CB) [11]

Hyperparameter Optimization with Optuna:

- Inner chronological validation
- Tree-structured Parzen Estimator (TPE) sampler [14]
- 20 trials per model and feature configuration
- Select best hyperparameters

Evaluation

Chronological 80/20 split:

- Preserve temporal order using TransactionDT
- Test set not used during training or optimization
- Benchmark: Kaggle Public and Private ROC-AUC

Evaluation Metrics

- Ranking Performance
 - ROC-AUC
 - PR-AUC [12]
- Classification Performance
 - Precision
 - Recall
 - F1-score
 - Balanced Accuracy
 - Matthews Correlation Coefficient (MCC)
 - Confusion Matrix

Overall performance

Top 5 Configurations by Local ROC-AUC

Model	Feature Set	Sampling	Local ROC-AUC	PR-AUC	Precision	Recall	F1	Bal. Acc.	MCC	Public	Private
XGBoost	Reduced FE	SMOTE	0.9217	0.5705	0.7987	0.3816	0.5165	0.6891	0.5421	0.9294	0.8981
LightGBM	Reduced Baseline	SMOTE	0.9217	0.5823	0.79	0.3989	0.5301	0.6975	0.5512	0.8639	0.8383
LightGBM	Feature Engineering	SMOTE	0.9208	0.5736	0.7929	0.3824	0.5159	0.6894	0.5405	0.8654	0.8185
XGBoost	Feature Engineering	Original	0.9196	0.5965	0.6805	0.5074	0.5813	0.7494	0.5752	0.9215	0.8865
XGBoost	Reduced Baseline	SMOTE	0.919	0.5614	0.7945	0.3767	0.5111	0.6866	0.537	0.9304	0.9015

Bottom 5 Configurations by Local ROC-AUC

Model	Feature Set	Sampling	Local ROC-AUC	PR-AUC	Precision	Recall	F1	Bal. Acc.	MCC	Public	Private
Decision Tree	Feature Engineering	SMOTE	0.7647	0.2607	0.1689	0.4235	0.2415	0.6746	0.2267	0.9071	0.8866
Decision Tree	Baseline	SMOTE	0.7902	0.2541	0.1348	0.499	0.2123	0.6924	0.2104	0.9073	0.8878
Decision Tree	Reduced Baseline	SMOTE	0.7963	0.2749	0.1544	0.4552	0.2306	0.6832	0.2212	0.9347	0.9052
Decision Tree	Feature Engineering	Original	0.7965	0.3368	0.1198	0.6754	0.2035	0.7493	0.2298	0.8383	0.8167
Decision Tree	Reduced FE	SMOTE	0.8136	0.3036	0.2794	0.4006	0.3292	0.6819	0.3062	0.9367	0.8997

Feature Engineering Effects

Metric	Without FE	With FE	Change
Local ROC-AUC	0.8775	0.8772	-0.0003
PR-AUC	0.4279	0.4288	0.0009
Precision	0.3316	0.336	0.0044
Recall	0.6203	0.6203	≈ 0
Kaggle Public ROC-AUC	0.9024	0.8997	-0.0027
Kaggle Private ROC-AUC	0.8648	0.8667	0.0019

Feature Reduction Effects

Metric	Original	Reduced	Change
Local ROC-AUC	0.8764	0.8783	0.0019
PR-AUC	0.43	0.4267	-0.0033
Precision	0.3289	0.3388	0.0099
Recall	0.6243	0.6163	-0.0080
Kaggle Public ROC-AUC	0.9031	0.899	-0.0041
Kaggle Private ROC-AUC	0.8583	0.8732	0.0149

Class Imbalance Handling Effects

Sampling	ROC-AUC	PR-AUC	Precision	Recall	F1	Bal. Acc.	MCC	Kaggle Public ROC-AUC	Kaggle Private ROC-AUC
Original	0.8797	0.4421	0.2741	0.6618	0.3487	0.7744	0.3607	0.9021	0.8613
SMOTE	0.8717	0.43	0.5628	0.4447	0.3985	0.6919	0.42	0.9008	0.8716
Random Undersampling	0.8807	0.4129	0.1646	0.7544	0.266	0.7977	0.3007	0.9023	0.8643

Model Comparison

Average ROC-AUC by Model

Model	Local ROC-AUC	Kaggle Public ROC-AUC	Kaggle Private ROC-AUC
Logistic Regression	0.8278	0.8551	0.792
Decision Tree	0.8136	0.8977	0.8736
Random Forest	0.8862	0.9222	0.8992
LightGBM	0.9163	0.8856	0.8378
XGBoost	0.9161	0.9207	0.8909
CatBoost	0.9043	0.929	0.901

Average Local Held-Out Test Performance by Model

Model	PR-AUC	Precision	Recall	F1	Bal. Acc.	MCC
LR	0.19	0.0882	0.7934	0.1587	0.7502	0.1975
DT	0.3011	0.1402	0.6069	0.2188	0.7289	0.2334
RF	0.4683	0.3916	0.5523	0.3634	0.7413	0.3829
LGBM	0.5478	0.4583	0.5943	0.4391	0.7737	0.4569
XGB	0.5541	0.5234	0.5627	0.468	0.7618	0.4823
CB	0.5088	0.4011	0.6122	0.3784	0.7723	0.4097

Error Analysis

XGBoost + Reduced Feature Engineering

Sampling	FP	FN	TP	Precision	Recall	F1
Original	1,593	1,899	2,165	0.5761	0.5327	0.5536
SMOTE	391	2,513	1,551	0.7987	0.3816	0.5165
Random Undersampling	11,315	990	3,074	0.2136	0.7564	0.3332

Limitations

- Single Dataset: limited to IEEE-CIS
- Temporal Validation
 - Single chronological holdout
 - No repeated temporal evaluation or confidence intervals
- Hyperparameter Optimization
 - Limited optimization budget
 - 20 Optuna trials per model and feature configuration
- Sampling & Threshold
 - Standard SMOTE applied after numerical encoding
 - Fixed classification threshold of 0.5

Future Work

- Evaluate additional datasets and time periods
- Rolling or walk-forward temporal validation
- Larger model-specific hyperparameter optimization
- Mixed-data or categorical-aware sampling
- Cost-sensitive learning and threshold optimization
- Explore graph-based models and GNNs

Conclusion

RQ1 - Feature Engineering

- uid, uid2, and DT_* produced limited aggregate improvement
- Effects varied across metrics and evaluation settings

RQ2 - Feature Reduction

- Removed approximately 21% of features
- Maintained similar local performance
- Improved Private Kaggle ROC-AUC

RQ3 - Model Comparison

- XGBoost and LightGBM showed strong local performance
- CatBoost showed strong Kaggle performance
- No single model dominated across all metrics and settings

RQ4 - Class Imbalance

- Sampling significantly changed the precision–recall trade-off
- SMOTE favored precision, F1-score, and MCC
- Random undersampling favored recall and balanced accuracy

Reference

- [1] A. Ali, S. Abd Razak, S. H. Othman, T. A. E. Eisa, A. Al-Dhaqm, M. Nasser, T. Elhassan, H. Elshafie, and A. Saif, “Financial fraud detection based on machine learning: A systematic literature review,” *Applied Sciences*, vol. 12, no. 19, 2022, doi: 10.3390/app12199637.
- [2] K. G. Dastidar, O. Caelen, and M. Granitzer, “Machine learning methods for credit card fraud detection: A survey,” *IEEE Access*, vol. 12, pp. 158939–158965, 2024, doi: 10.1109/ACCESS.2024.3487298.
- [3] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [4] H. Peng, F. Long, and C. Ding, “Feature selection based on mutual information: Criteria of max-dependency, max-relevance, and min-redundancy,” *Pattern Recognition*, vol. 38, no. 8, pp. 1226–1238, 2005, doi: 10.1016/j.patcog.2005.01.019.
- [5] P. A. Estévez, M. Tesmer, C. A. Perez, and J. M. Zurada, “Normalized mutual information feature selection,” *IEEE Transactions on Neural Networks*, vol. 20, no. 2, pp. 189–201, 2009, doi: 10.1109/TNN.2008.2005601.
- [6] J. R. Vergara and P. A. Estévez, “A review of feature selection methods based on mutual information,” *Neural Computing and Applications*, vol. 24, no. 1, pp. 175–186, 2014, doi: 10.1007/s00521-013-1368-0.
- [7] F. Moradi, M. Tarif, and M. Homaei, “Robust fraud detection with ensemble learning: A case study on the IEEE-CIS dataset,” *Preprints.org*, Jul. 2025, doi: 10.20944/preprints202507.1711.v1. [Not peer reviewed].
- [8] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.
- [9] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, ACM, 2016, doi: 10.1145/2939672.2939785.
- [10] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems*, vol. 30, pp. 3146–3154, 2017.
- [11] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, “CatBoost: Unbiased boosting with categorical features,” in *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [12] T. Saito and M. Rehmsmeier, “The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets,” *PLOS ONE*, vol. 10, no. 3, Art. no. e0118432, 2015, doi: 10.1371/journal.pone.0118432.
- [13] A. Howard, B. Bouchon-Meunier, C. Huang, inversion, J. Lei, Lynn@Vesta, Marcus2010, and H. Abbass, “IEEE-CIS Fraud Detection,” *Kaggle*, 2019. [Online]. Available: IEEE-CIS Fraud Detection competition
- [14] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, “Algorithms for hyper-parameter optimization,” in *Advances in Neural Information Processing Systems*, vol. 24, 2011.

Thank you

